

Artificial Intelligence 2025

Homework: Solving the N-Queens Problem

Search vs. Constraint Satisfaction

Francesco Maria Germano, 2212589

Contents

1	Introduction	2
2	Problem Formulation	2
2.1	State space representation	3
3	Method 1: A* Search Algorithm	3
3.1	Heuristics	3
3.1.1	Heuristic h_1 : number of attacking pairs (traditional heuristic used in the AI class)	3
3.1.2	Heuristic h_2 : aggressive penalty (weighted heuristic)	4
4	Method 2: Constraint Satisfaction Problem (CSP)	4
4.1	Encoding 1: pairwise diagonals (binary constraints)	4
4.2	Encoding 2: AllDifferent with auxiliary variables	4
5	Experimental Results	5
5.1	A* performance analysis	5
5.2	CSP encodings comparison	7
5.3	Search vs. CSP: final verdict	9
6	Conclusion	10

1 Introduction

The N-Queens problem consists of placing N chess queens on an $N \times N$ chessboard so that no two queens threaten each other. Accordingly to the rules of each piece of chess, this means that no two queens can share the same row, same column, or same diagonal.

While the problem can be solved for small N using brute force, the complexity grows factorially, making it an ideal study for evaluating ways to solve it more efficiently with what has been studied in the Artificial Intelligence class. The assignment requires to implement two ways to solve the same problem, then make a comparison:

1. **Informed Search:** Implementing the A* algorithm with some different heuristic functions (described more specifically later) to guide the exploration of the state space.
2. **Constraint Satisfaction (CSP):** Modeling the problem as a set of variables and constraints, leveraging propagation and backtracking.

This report is organized as follows:

- Section 2 formalizes the problem.
- Section 3 involves the A* implementation and heuristics.
- Section 4 describes the CSP modeling and encodings.
- Section 5 presents the experimental setup and results.
- Section 6 draws conclusions on the two approaches.

2 Problem Formulation

The N-Queens problem can be formally defined depending on the approach used. However, the fundamental rules remain the same. The scaling parameter of this problem is N . Given an $N \times N$ grid, N queens (the only piece that can move in any direction) have to be placed such that:

- **Row constraint:** $\forall i, j \in \{1, \dots, N\}, i \neq j \implies \text{row}(q_i) \neq \text{row}(q_j)$
- **Column constraint:** $\forall i, j \in \{1, \dots, N\}, i \neq j \implies \text{col}(q_i) \neq \text{col}(q_j)$
- **Diagonal constraint:** $|\text{row}(q_i) - \text{row}(q_j)| \neq |\text{col}(q_i) - \text{col}(q_j)|$

2.1 State space representation

For the search algorithm, the following formulation has been adopted:

- **States:** A state is represented by a partial arrangement of k queens on the first k columns, where $0 \leq k \leq N$.
- **Initial state:** An empty board ($k = 0$).
- **Actions:** Placing a queen in the leftmost empty column such that it does not attack any previously placed queen.
- **Goal state:** N queens are placed on the board, and no conflicts exist.
- **Path cost:** The cost is uniform; each step costs 1.

3 Method 1: A* Search Algorithm

A* is a best-first search algorithm that finds the least-cost path from a given initial node to a goal node. The difference with Dijkstra algorithm is the presence of a heuristic function (that gets summed to the actual path cost), in the cost function denoted as $f(n)$:

$$f(n) = g(n) + h(n) \tag{1}$$

where $g(n)$ is the cost from the start node to node n , and $h(n)$ is the estimated cost from node n to the goal, accordingly to the adopted variant.

In the implementation, since the depth of the solution is fixed at N , optimizing $g(n)$ is trivial. The crucial component is the heuristic $h(n)$.

3.1 Heuristics

Two heuristics have been implemented and compared, to evaluate their impact on the branching factor and search space.

3.1.1 Heuristic h_1 : number of attacking pairs (traditional heuristic used in the AI class)

This heuristic relaxes the problem constraints. It counts the number of pairs of queens that are currently attacking each other in the partial state.

$$h_1(n) = \text{count of pairs } (q_i, q_j) \text{ s.t. } q_i \text{ attacks } q_j$$

Although simple, this heuristic provides a baseline for performance, representing a direct measure to the problem constraints violation.

3.1.2 Heuristic h_2 : aggressive penalty (weighted heuristic)

This heuristic simply takes the conflicts number and multiplies it for a scaling factor (set to 10). This heuristic transforms the search behaviour, by making the algorithm escape from the conflict zones, rather than directly looking for the optimal path.

$$h_2(n) = 10 \times (\text{count of pairs } (q_i, q_j) \text{ s.t. } q_i \text{ attacks } q_j)$$

This easily makes the $h(n)$ value dominant with respect to the path cost $g(n)$, this means that the expansion strategy will lead to a more aggressive pruning.

4 Method 2: Constraint Satisfaction Problem (CSP)

A Constraint Satisfaction Problem is defined by a triplet (V, D, C) :

- $V = \{v_0, v_1, \dots, v_{N-1}\}$ is the set of variables, where v_i represents the row of queen in i -th column.
- $D = \{1, 2, \dots, N\}$, for all $v \in V$
- C is the set of constraints specifying allowable combinations of values for $1 \leq i \leq N$:

$$\text{Row Constraint: } X_i \neq X_j \quad \forall i \neq j \quad (2)$$

$$\text{Diagonal Constraint: } |X_i - X_j| \neq |i - j| \quad \forall i \neq j \quad (3)$$

The CSP solver has been implemented using the `python-constraint` library, testing two different encodings for the diagonal constraints.

4.1 Encoding 1: pairwise diagonals (binary constraints)

This formulation adheres to the definition of the problem using only the N variables representing the columns. The diagonal constraints are expressed as binary constraints between every pair of variables (v_i, v_j) :

$$|v_i - v_j| \neq |i - j| \quad \forall i < j \quad (4)$$

This approach generates $O(N^2)$ constraints. While the number of constraints is high, they are simple checks that the solver can propagate efficiently.

4.2 Encoding 2: AllDifferent with auxiliary variables

This encoding attempts to reduce the number of constraint checks by introducing auxiliary variables. For any square (r, c) , the diagonals are constant values:

- Main diagonal index: $d_1 = r - c$
- Anti-diagonal index: $d_2 = r + c$

$2N$ auxiliary variables are introduced, representing these diagonal indices and a global `AllDifferent` constraint is applied on them. While theoretically elegant ($O(N)$ constraints), this approach increases the number of variables in the CSP from N to $3N$.

5 Experimental Results

The experiments were conducted on a machine with Intel i7 10-th generation with 16 cores, 16 GB RAM. A timeout limit has been set to the single run of an algorithm: 300 seconds per instance, which treated the complexive run as a "survival race", where a heuristic exceeding the given time got deactivated, although reporting the time taken also for that instance.

The scaling parameter N ranged from 4 to 50. Further tables show how these limits could not be always reached, with earlier termination for some instances.

5.1 A* performance analysis

As expected for an NP-complete problem, A* struggles as N increases. The metrics adopted are the following:

- Time taken;
- Nodes expanded (the ones extracted from the frontier);
- Nodes generated (nodes added to the frontier);
- Max memory (max number of states simultaneously held in memory);
- Solution cost (cost of $g(n)$ from initial to the goal state);
- Solution depth;
- Branching factor (average of branching factors across the whole instance run).

The following table summarizes the results obtained, numerically:

Alg	Heuristic	N	Time (s)	Nodes exp.	Nodes gen.	Max mem.	Cost	B. factor
A*	h1	4	0.0008	48	189	189	4	3.9375
A*	h2	4	0.0002	16	61	61	4	3.8125
A*	h1	5	0.0054	222	1,106	1,106	5	4.9820
A*	h2	5	0.0007	45	221	221	5	4.9111
A*	h1	6	0.0363	1,280	7,675	7,675	6	5.9961
A*	h2	6	0.0049	150	895	895	6	5.9667
A*	h1	7	0.3081	8,579	60,047	60,047	7	6.9993
A*	h2	7	0.0137	513	3,585	3,585	7	6.9883
A*	h1	8	2.2818	61,486	491,881	491,881	8	7.9999
A*	h2	8	0.0642	1,966	15,721	15,721	8	7.9964
A*	h1	9	27.3890	491,337	4,422,025	4,422,025	9	9.0000
A*	h2	9	0.3502	8,043	72,379	72,379	9	8.9990
A*	h1	10	933.0583	4,285,102	42,851,011	42,851,011	10	10.0000
A*	h2	10	3.3318	34,816	348,151	348,151	10	9.9997
A*	h2	11	15.5846	164,247	1,806,707	1,806,707	11	10.9999
A*	h2	12	134.9579	842,024	10,104,277	10,104,277	12	12.0000

Table 1: Detailed experimental results for A* experiments. Time and Branching Factor are rounded to 4 decimal places.

A graphical analysis follows, to depict the values obtained in a visual manner.

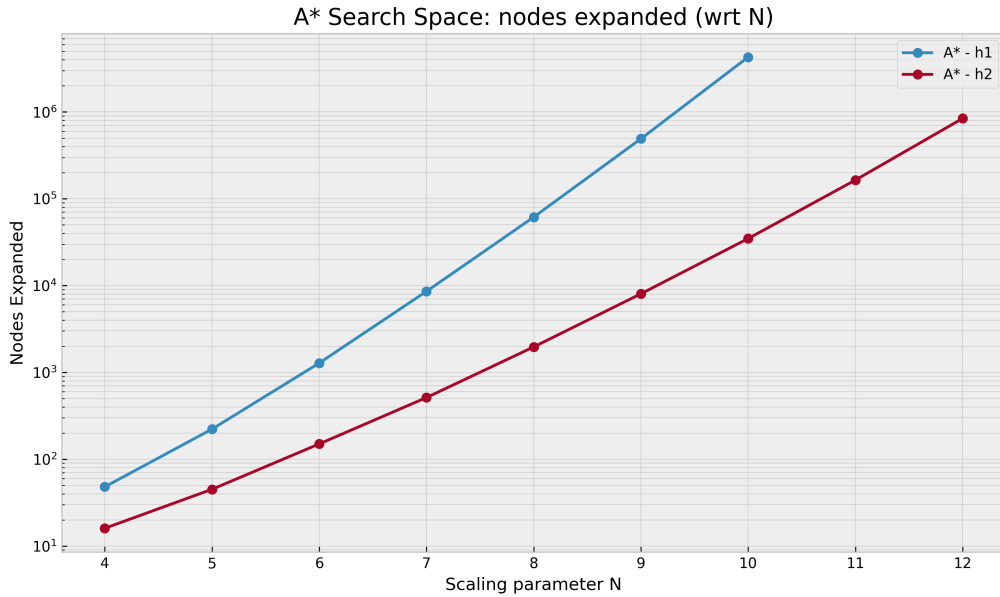


Figure 1: Nodes expanded by A* (Log Scale). The exponential growth is evident.

Figure 1 clearly shows the exponential explosion of the search space.

- **Impact of Heuristics:** h_2 consistently outperforms h_1 , expanding fewer nodes to find the solution. This confirms that a more aggressive heuristic significantly prunes the search tree.
- **Limit:** Despite the heuristic, A* fails to solve instances larger than $N \approx 12$ within the time limit.

5.2 CSP encodings comparison

Comparing the two CSP encodings reveals an interesting insight into solver overhead. The metrics of interest adopted are the following:

- Time taken;
- Variables count ($|V|$);
- Constraints count ($|C|$);

The following table summarizes the results obtained, numerically:

Alg	N	Encoding	Time (s)	Vars	Constraints
CSP	4	pairwise	0.000183	4	7
CSP	4	alldiff	0.002142	12	11
CSP	5	pairwise	0.000184	5	11
CSP	5	alldiff	0.002249	15	13
CSP	6	pairwise	0.000696	6	16
CSP	6	alldiff	0.025602	18	15
CSP	7	pairwise	0.000253	7	22
CSP	7	alldiff	0.037971	21	17
CSP	8	pairwise	0.001705	8	29
CSP	8	alldiff	0.502609	24	19
CSP	9	pairwise	0.001148	9	37
CSP	9	alldiff	1.560339	27	21
CSP	10	pairwise	0.001161	10	46
CSP	10	alldiff	10.916482	30	23
CSP	11	pairwise	0.001647	11	56
CSP	11	alldiff	94.793424	33	25
CSP	12	pairwise	0.006884	12	67
CSP	12	alldiff	1389.695882	36	27
CSP	50	pairwise	0.190651	50	1226

Table 2: Experimental results for CSP solver: Time complexity and structural metrics vs N.

A graphical analysis follows, to depict the values obtained in a visual manner.

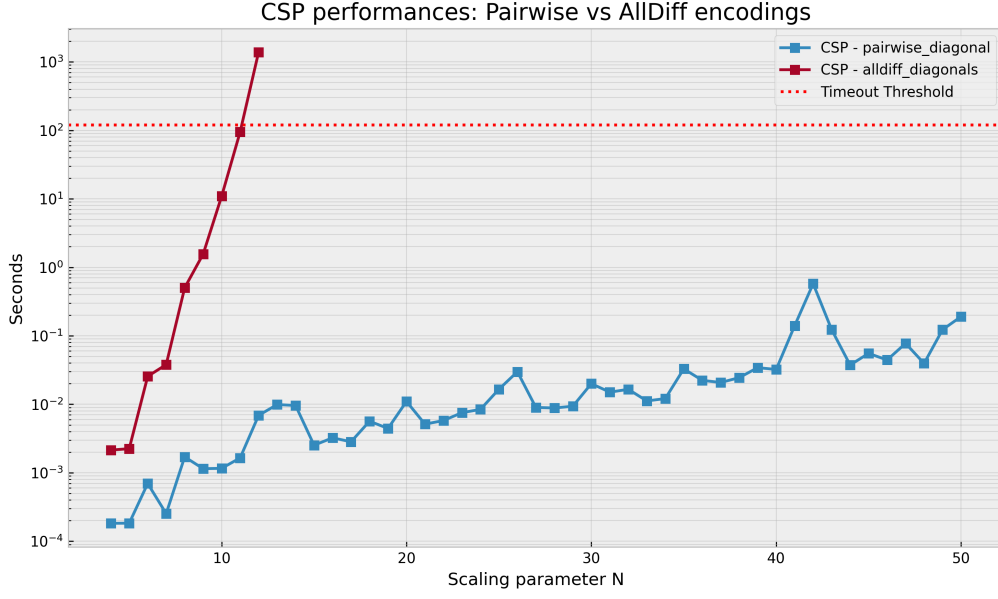


Figure 2: Comparison of CSP Encodings: Pairwise vs. AllDiff.

As shown in Figure 2, the *Pairwise* encoding is orders of magnitude faster than the *AllDiff* encoding.

- **Pairwise:** Maintains the variable count at N . The solver handles the binary constraints efficiently.
- **AllDiff:** The overhead of managing $2N$ extra auxiliary variables outweighs the benefit of having fewer global constraints in the Python implementation.

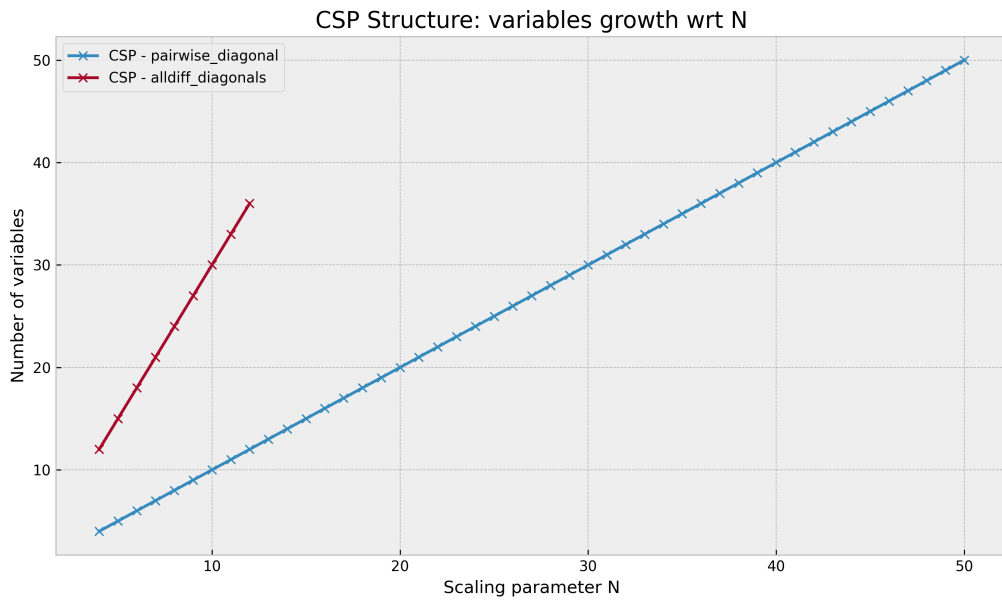


Figure 3: CSP Encodings: growth of variables.

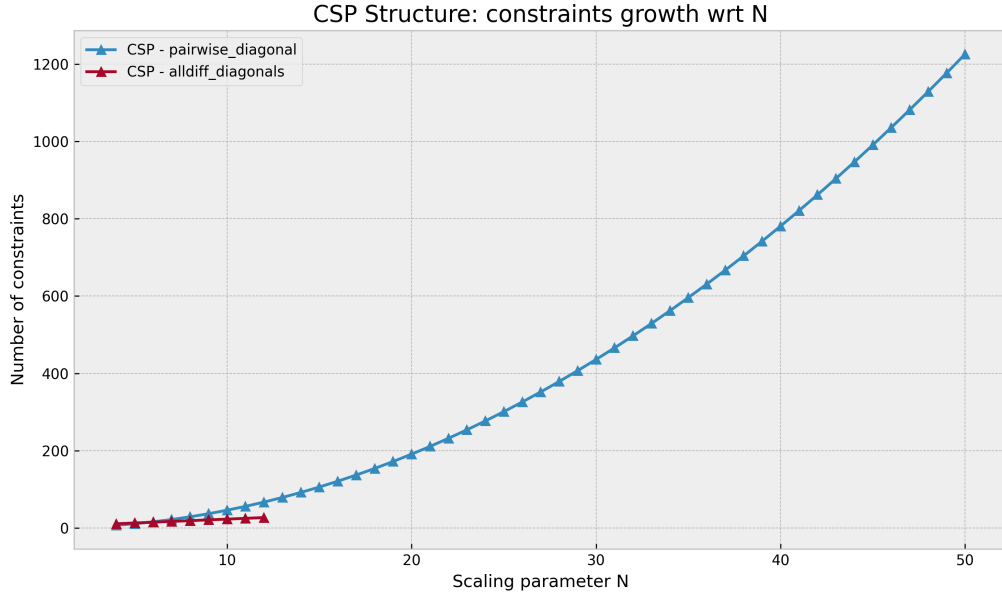


Figure 4: CSP Encodings: growth of constraints.

5.3 Search vs. CSP: final verdict

As requested, this section shows the comparison among the two adopted techniques.

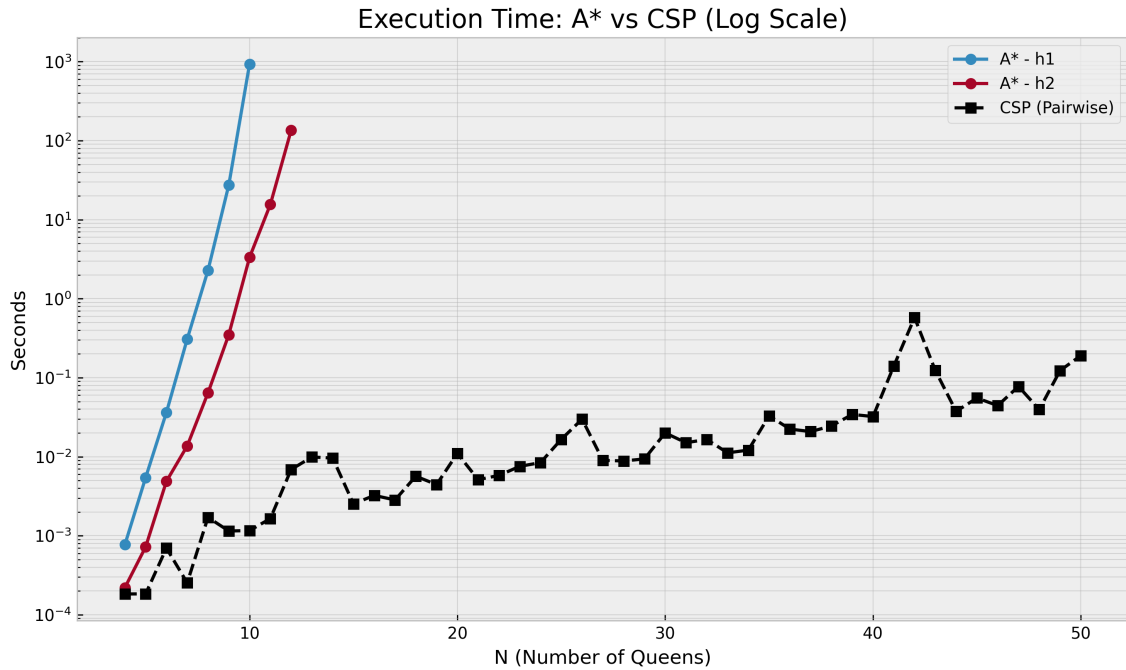


Figure 5: Execution Time Comparison: A* vs CSP (Log Scale).

CSP solves $N = 50$ in less than a second, while A* times out at $N = 12$. This demonstrates that for combinatorial assignment problems with hard constraints, inference (CSP) is vastly superior to search (A*). The ability of CSP to propagate constraints and prune inconsistent

subtrees without explicit expansion makes it the most suitable way to solve for N-Queens (among the two experimented in this assignment).

6 Conclusion

In this homework, the N-Queens problem has been explored through Search and CSP. The results prove the theoretical expectations. After an initial, simpler implementation of CSP, a more complex one has followed, enhancing the final version of this assignment, namely the one being presented:

1. **Search limitations:** A* is effective for pathfinding but inefficient for large combinatorial arrangement problems due to the factorial state space.
2. **Heuristic importance:** Better heuristics (h_2) delay the exponential explosion but cannot prevent it.
3. **CSP dominance:** Constraint Satisfaction is the correct approach for this class of problems.
4. **Implementation matters:** A simpler encoding (Pairwise) proved superior to a theoretically compact one (AllDiff) due to the overhead of auxiliary variables in the specific solver implementation.